Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
benjamin_houle@student.uml.edu

Ebenezer Oloyede
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Ebenezer_Oloyede@student.uml.edu

Lily Grippo
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Lily$_{G}rippo@student.umle.edu$

Name Surname
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
email address or ORCID

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations for information control flow and memory allocations. Utilizing the well-supported Manim library alongside LLVM's built-in static analysis tools and wide language support, LLVManim is able to quickly create informative, professionally styled animations for any code that compiles to LLVM IR. The primary objective of the project is to create an alternative to the unappealing "programmer art" and visual clutter that many standard tools suffer from, instead focusing on an output more suitable for presentations or educational purposes.**

***CCS Concepts:* • TODO (non-critical): - https://dl.acm.org/ccs**

***Index Terms*—TODO (non-critical):**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, it seems that disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating light-weight readable output, assuming the user is more concerned with the information than its presentation. This, however, is not always the case: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually appealing content opposed to static, large blocks of text. In these cases, the presenter is currently forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves.

II. Existing Tools

A. PythonTutor

PythonTutor is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, PythonTutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [**?**].

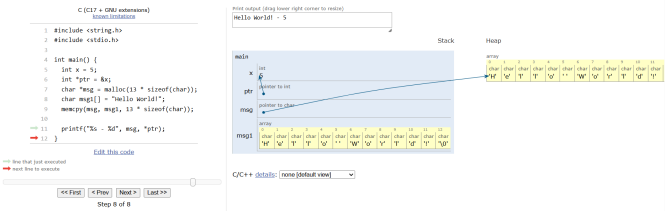

Fig. 1. Example Pythontutor.com output.

In fact, Pythontutor has been shown to be effective in improving students' understanding of programming concepts; Guo's further studies demonstrated significant learning gains when Pythontutor was used as a supplement to traditional instruction [**?**]. This shows the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. PythonTutor's primary focus is on clarity and educational value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-based and not designed for offline use, makes it less than ideal for use in settings such as conference presentations where visual appeal is important. PythonTutor is additionally restricted by its support for only six programming languages, limited customization options, and minimal library of visualization methods.

B. LLVM Analysis and Transform Passes

LLVM has emerged as the de-facto standard compiler infrastructure for modern software development, powering everything from the development environment for Apple operating systems to cutting-edge systems research efforts. Its adoption by the compiler community can be attributed to

several key strengths$dashrobust support for numerous programming languages including C, C++, Rust, Swift, and Julia; a modular architecture that separates language frontends from optimization backends; and an extensive suite of analysis and transformation passes that expose deep insights into program behavior [**?**].

At the core of LLVM's value proposition for code analysis is its Intermediate Representation (IR), a low-level yet platform-independent format that captures program semantics in a standardized way. This IR serves as the foundation for powerful built-in analysis passes that can generate control flow graphs, dominator trees, data flow analyses, and detailed memory access patterns. For compiler developers and researchers, these capabilities are invaluable—LLVM effectively solves the hard problem of extracting structured information from code, allowing them to focus on higher-level tasks like optimization or verification [**?**].

However, LLVM's visualization capabilities tell a different story. The standard approach for visualizing analysis results relies on generating DOT files through passes like `-dot-cfg` or `-view-cfg`, which are then rendered using Graphviz. While functionally correct, the resulting diagrams embody a strictly utilitarian aesthetic: monochrome nodes and edges, densely packed labels filled with LLVM IR instructions, and layouts optimized for completeness rather than clarity. This "function over form" philosophy serves well enough as a developer tool for debugging optimizer passes, but it fails catastrophically as a solution for communication beyond that narrow domain. A lecturer explaining recursion to undergraduates, a tech lead presenting architectural decisions to stakeholders, or a researcher delivering a conference talk all face the same problem: LLVM-generated visualizations are too dense, too technical, and frankly, too ugly to serve their pedagogical or presentational needs.

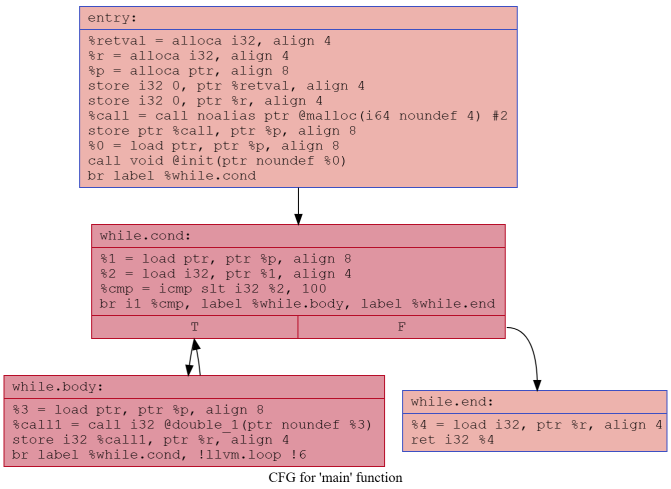

Fig. 2. Example Control Flow Graph using LLVM `-dot-cfg` pass. While technically accurate, the visualization prioritizes information density over visual clarity.

The barrier extends beyond mere aesthetics. Interpreting these diagrams requires familiarity with LLVM IR syntax—understanding what "`%4 = load i32, i32* %3`" means demands compiler knowledge that most audiences lack. The visualizations are fundamentally static images with no mechanism for progressive disclosure or animation, forcing presenters to mentally simulate execution flow while pointing at a frozen graph. Moreover, customization options are severely limited; changing colors, adjusting layouts, or highlighting specific execution paths requires manual post-processing in external tools, completely defeating the purpose of automated generation.

In essence, LLVM provides extraordinarily powerful analysis capabilities but pairs them with visualization tools that assume their primary consumer is someone who already understands what they're looking at. This creates a frustrating gap: we can automatically extract rich semantic information about program behavior, but we cannot automatically present that information in a way that serves educational or presentational contexts. LLVManim aims to bridge this gap by maintaining LLVM's analytical rigor while replacing its austere output with animations designed for human communication rather than machine precision.

III. Manim

The Manim library was originally developed by Grant Sanderson for his educational YouTube channel "3Blue1Brown", specifically for animating math and computer science concepts. His beginner-friendly animations assisted him in becoming one of the more well-known online personalities in these fields and have helped countless others improve their understanding of complex topics. This history makes it a prime candidate as a tool for creating high-quality educational/informational presentations, but many are unable or unwilling to spend the time and effort it takes to create custom animations using a third-party Python library.

While tools like PythonTutor and LLVM address the technical challenge of extracting and displaying program information, they fall short in making that information genuinely engaging. This is where Manim distinguishes itself, not as a code analysis tool, but as a demonstration of what is possible when visualization prioritizes both clarity and aesthetic appeal.

Since its inception in 2015, the 3Blue1Brown channel has gained more than 6 million subscribers and hundreds of millions of views. Individual videos on topics such as linear algebra, calculus, and even neural networks regularly reach audiences of millions. Sanderson's approach, using smooth, precisely choreographed animations to build intuition about abstract concepts, has earned him widespread recognition in the educational community; he won a 2020 YouTube Streamy Award for Education. The key to all of these animations is Manim, an animation engine written in Python and maintained by Sanderson himself.

Manim's strength lies in its ability to programmatically generate animations with mathematical precision. Instead of tiresome keyframe animation like other software, Manim allows users to write Python code to determine how different

mathematical objects should look, change, and interact with each other. Manim can render beautiful 3D scenes, render LaTeX equations, and output crisp 60 fps animations for any purpose, from YouTube tutorials to academic conferences. What makes Manim unique for education is its capacity to make complicated ideas, like matrix transformations, function composition, or even program flow, tangible and understandable to the viewer by systematically unfolding them.

Yet, this power comes with a barrier to entry. It is not trivial for a professor who wants to make a presentation for a single lecture, or a researcher who wants to create visualizations for a conference presentation, to learn and use Manim effectively. They may have a clear idea of what they want to show, perhaps how a recursive function builds up its stack, but writing Manim code for it requires a distinct skill set. LLVManim addresses this challenge by automatically translating program semantics into visual representations, thereby eliminating the barrier of manual coding between LLVM's analytical power and Manim's visual sophistication.

IV. LLVMANIM

By providing a solution to quickly and easily create data-based animations, LLVManim aims to bridge the gap between these two concepts:

1) Methods exist to quickly generate code visualizations, but they are unappealing for presentation purposes.
2) The Manim library can make visually impressive animations, but using it is time consuming.

Educators and tech leads alike can rejoice in spicing up their PowerPoints without needing to compromise between style and accessibility.

A. Project Goals

The primary goal of LLVManim is to create a lightweight pipeline for converting LLVM IR and runtime traces into concise, animated explanations of program behavior. As a proof-of-concept, the initial version of LLVManim will focus on visualizing fundamentals, such as control flow and memory allocations.

Proof-of-Concept Targets:

- **IR Instruction Coverage:** Support for the most common LLVM IR instructions, such as `load`, `store`, `call`, and basic arithmetic operations. This will allow LLVManim to visualize a wide range of simple programs.
- **Trace Event Mapping:** Correctly map $\geq 95\%$ of runtime events (loads, stores, branches, calls, returns) to IR locations when debug metadata is present.
- **Customization Options:** Provide ≥ 4 user-configurable options – Analysis method, animation speed, color palette, & verbosity.
- **Usability:** In a pilot study (6–8 participants), achieve $\geq 80\%$ task completion for locating injected bugs.

Core Features:

- **LLVM IR Parsing:** Extract control flow and memory access information from LLVM IR, utilizing debug metadata when available.
- **Manim Animation Generation:** Translate LLVM IR and trace information into Manim scenes that visually represent control flow, stack frames, heap allocations, and data flow.
- **User Interface:** Develop a simple command-line interface that allows users to specify input files, select visualization options, and generate animations with minimal configuration.
- **Basic Animation Customization:** Allow users to customize the appearance of animations (e.g., color schemes, animation speed, etc.) to suit different presentation contexts.
- **TODO (non-critical):** Come up with more core features.

Major features (required to realize the intended product):

- **Advanced Animation Customization:** Allow users to customize more aspects of the animations, such as layout styles, node shapes, or animation effects.
- **Basic Interactive Features:** Implement basic interactivity in the generated animations, such as pausing, stepping through execution, or highlighting specific paths.
- **TODO (non-critical):** Come up with more major features.

Stretch features (nice-to-have if time permits):

- **GUI:** Create a graphical user interface for users who prefer not to use the command line.
- **Advanced Interactive Features:** Implement advanced interactivity in the generated animations, such as allowing users to click on nodes to see underlying IR code or variable values.
- **TODO (non-critical):** Come up with more stretch features.

B. Implementation Plan

TODO (critical): Explain approach and details for how the tool will operate, present UI mock-up.

V. PROJECTED TIMELINE

Week	Task
W_1 [2/08 – 2/14]	• Project proposal draft complete. • Development environment setup.
W_2 [2/15 – 2/21]	•
W_3 [2/22 – 2/28]	•
W_4 [3/01 – 3/07]	•
W_5 [3/08 – 3/14]	•
W_6 [3/15 – 3/21]	•
W_7 [3/22 – 3/28]	•
W_8 [3/29 – 4/04]	•
W_9 [4/04 – 4/10]	•
W_{10} [4/11 – 4/17]	•

TABLE I
PROJECTED TIMELINE FOR LLVMANIM DEVELOPMENT.

VI. Feedback

”The figures are helpful for showing how the existing tools work, but for the proposal make sure you show how you want the visualized code to work. This could be through making a mockup or setting up input for Manim to generate something that looks like what you want. You should also think about the pipeline for what data you are planning on getting from LLVM, how you are going to get it, and how you will feed it into Manim.

You have figures for some existing tools. Make sure you talk about them in the proposal and what their limitations are.”

TODO (critical): describe any feedback you did NOT take action on, with a brief justification for why (for example, if some other project related to your proposal was suggested, but it is not, this is where you would say why it was not related). Include ALL feedback you have received on the project at any point, whether from the instructor or your peers. If you have addressed every piece of feedback and made changes in response, write “We have addressed all feedback.”

Acknowledgment

The preferred spelling of the word “acknowledgment” in America is without an “e” after the “g”. Avoid the stilted expression “one of us (R. B. G.) thanks ...”. Instead, try “R. B. G. thanks...”. Put sponsor acknowledgments in the unnumbered footnote on the first page.